

Reviewer Pack — Minimal Rank of Algebraic Stacks over Boolean Functions vs R...

Entry b5838f9502ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 16:10:20 UTC

Minimal Rank of Algebraic Stacks over Boolean Functions vs Randomized Circuit Complexity

Entry ID: b5838f9502ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 16:10:20 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Stacks) **Field B** (complexity object): Complexity Theory: Randomized Circuit Complexity

Statement:

[‘For a given Boolean function f with n variables, the minimal rank of its algebraic stack A_f is proportional to the logarithm of the randomized circuit complexity $R(f)$.’, ‘Equivalently, for any constant $c > 0$, there exists a polynomial $p(n)$ such that $\log(R(f)) \leq c * \text{MinRank}(A_f) * \log(n)$.’, ‘Furthermore, the ratio $\text{MinRank}(A_f) / \log(n)$ approaches zero as n grows.’]

Rationale (proposer’s reasoning):

[‘Algebraic stacks provide a higher-dimensional framework for studying algebraic varieties and their geometric properties. Their minimal rank might capture essential features of Boolean functions that are not visible in lower-dimensional structures.’, ‘Randomized circuit complexity measures the difficulty of computing a function with a small number of random queries. A direct relationship between these two could imply a deep connection between algebraic geometry and computational complexity.’]

Taxonomy category: algebraic_stacks_randomized_circuit_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): df1d60f64cd53c89

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all generated Boolean functions f with $n \leq 40$ satisfy $\text{MinRank}(A_f) * \log(n) \geq c * \log(R(f))$ for at least 80% of seeds, where c is a constant > 0 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic stack" AND "randomized circuit complexity" - "minimal rank" AND "Boolean function" AND "polynomial growth" - "algebraic geometry" AND "complexity theory" AND "logarithmic relationship"

Top relevant hits considered: - [s2:2106.05305] Linear growth of quantum circuit complexity - [s2:2501.12260] Nonuniform Deterministic Finite Automata over finite algebraic structures - [s2:3540928c663bdf104f338ea4908330fc8f9f18cf] Configuration spaces : geometry, combinatorics and topology

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import product

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k, n = len(A), len(B), len(B[0])
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    Augmented = [A[i] + [b[i]] for i in range(m)]
```

```

for i in range(n):
    max_row = i
    for j in range(i+1, m):
        if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
            max_row = j
    Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
    pivot = Augmented[i][i]
    for j in range(i, n + 1):
        Augmented[i][j] /= pivot
    for j in range(m):
        if j != i:
            factor = Augmented[j][i]
            for k in range(i, n + 1):
                Augmented[j][k] -= factor * Augmented[i][k]
return [row[-1] for row in Augmented]

def min_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(min(m, n)):
        if matrix[i][i] != 0:
            rank += 1
    return rank

def algebraic_stack(f):
    n = len(f)
    A = [[0] * (n + 1) for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = f[i][j]
        A[i][n] = 1
    return A

def randomized_circuit_complexity(f):
    n = len(f)
    if n == 1:
        return 0
    complexity = 0
    for _ in range(10): # Sample 10 random inputs
        x = [random.choice([0, 1]) for _ in range(n)]
        y = f(x)
        if y not in [0, 1]:
            return float('inf')
        complexity += 1
    return complexity

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        f = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        A = algebraic_stack(f)
        R = randomized_circuit_complexity(f)
        if R == float('inf'):
            continue

```

```

min_rank_A = min_rank(A)
results.append({
    "n": n,
    "min_rank_A": min_rank_A,
    "R": R
})
metric_value = sum(result["min_rank_A"] * math.log(result["n"]) for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["min_rank_A"] * math.log(result["n"]) >= 10 * math.log(result["n"]) for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"
return {
    "metric_name": "MinRank(A_f) * log(n)",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unsupported_operation")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e53493f.py", line 116, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e53493f.py", line 90, in run_trial
    R = randomized_circuit_complexity(f)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e53493f.py", line 77, in randomized_circuit_complexity
    y = f(x)

```

^^^^

TypeError: 'list' object is not callable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered support condition.
| next: Re-run the test with proper error handling and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12203
2	propose	ollama_remote	glm4:latest	0	0	10297
3	preregistration	ollama_remote	glm4:latest	0	0	5872
4	novelty	ollama_remote	glm4:latest	0	0	4930
5	novelty	ollama_remote	glm4:latest	0	0	5756
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35648
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9424
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10645
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14122
10	judge	ollama_remote	glm4:latest	0	0	30084

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138982 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b5838f9502ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b5838f9502ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b5838f9502ac.tar.gz` (if generated)